

ER SCHEMA : Generalization, Features Layer , Aggregation

Why Booking NEEDS Generalization

Look at your three booking types. A transport booking needs `departure_id`, `num_seats`. A hotel booking needs `checkin_date`, `checkout_date`, `num_rooms`. A package booking needs `package_departure_id`, `num_travellers`.

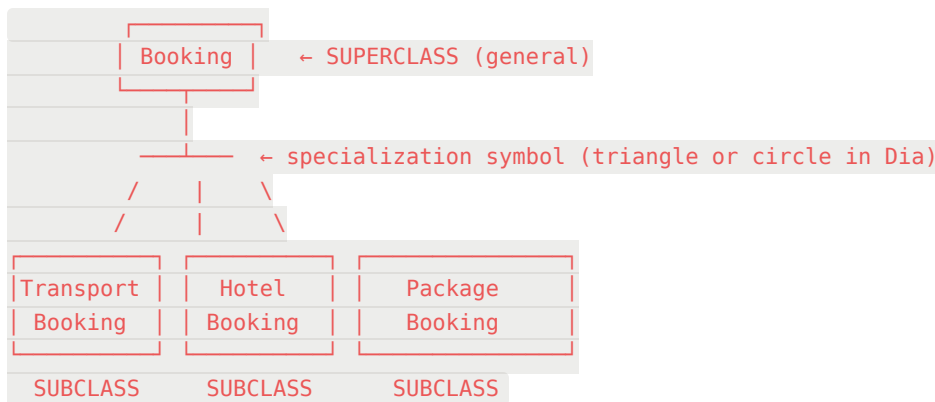
These three share common attributes — `booking_id`, `user_id`, `status`, `final_amount`, `payment_method`, `booked_at`. But each has completely different specific attributes that don't apply to the others.

This is **exactly** the situation generalization/specialization was invented for. You cannot find another clean way to model this — if you put everything in one table you get nulls everywhere. This is your justified use.

The Concept — Superclass and Subclass

Superclass = the general entity that holds everything common.

Subclass = the specialized entity that holds what's unique to that type.



Two Properties You Must Decide

① Disjoint vs Overlapping

Disjoint (d) — one booking can only be ONE type. A booking is either transport OR hotel OR package. Never two at once.

Overlapping (o) — one booking could be multiple types simultaneously.

Your system → **Disjoint**. One booking_id is strictly one type. Write "**d**" inside the circle/triangle in Dia.

② Total vs Partial Specialization

Total — every instance of the superclass MUST belong to at least one subclass. Every booking MUST be either transport, hotel, or package. There is no "generic" booking that belongs to nothing.

Partial — some instances of the superclass might not belong to any subclass.

Your system → **Total**. Every single booking record must be one of the three types. Write "**Total**" or use double line from Booking down to the triangle.

What Each Subclass Contains

Booking (Superclass)

```
booking_id      PK
user_id         FK
booked_at
status
base_amount
discount_amount
final_amount
promo_id        FK
payment_method
payment_ref
booking_type    (discriminator attribute)
```

The `booking_type` attribute is called the **discriminator** — it tells you which subclass this row belongs to. In Dia, mark it with a dashed underline or note it separately.

TransportBooking (Subclass)

```
Inherits all Booking attributes PLUS:
departure_id    FK → ScheduleDeparture
num_seats
```

HotelBooking (Subclass)

```
Inherits all Booking attributes PLUS:
hotel_id         FK → Hotel
room_category_id FK → RoomCategory
checkin_date
checkout_date
num_rooms
num_guests
```

PackageBooking (Subclass)

Inherits all Booking attributes PLUS:
package_departure_id FK → PackageDeparture
num_travellers

Passenger connects to the Subclass, not the Superclass

This is important. `Passenger` is a weak entity under `TransportBooking` specifically — not under the general `Booking`. So the `has_passenger` relationship connects to the `TransportBooking` subclass rectangle, not to the `Booking` superclass.

Similarly, `PackageTraveller` (the list of people on a package) connects to `PackageBooking` subclass.

`TransportBooking` — `has_passenger` — `Passenger`
`PackageBooking` — `has_traveller` — `PackageTraveller`

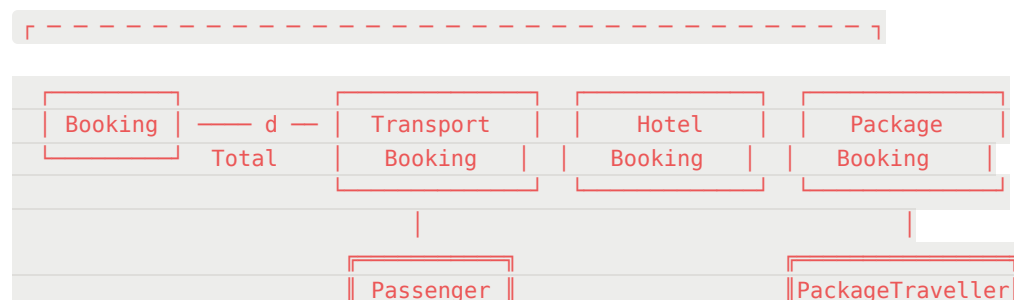
What is Aggregation here specifically?

When a User books a ScheduleDeparture, that entire booking event — involving User, ScheduleDeparture, payment, status — is captured as `Booking`. Now Insurance doesn't connect to just the User. It doesn't connect to just the ScheduleDeparture. It connects to the **entire booking event as one thing**.

In your Image 2 your professor drew a box around the relationship to treat it as an entity. That box is exactly `Booking` in your system. You've already done the aggregation — `Booking` is the aggregated entity. Now Insurance, CancellationRefund and Review all hang off that aggregated entity.

he Aggregation Box

In your Dia diagram, draw a **large dashed rectangle** that encloses the entire booking cluster:



Complete Entity + Relationship Count

Category	Count	Names
Strong Entities	8	User, Vendor, TransportRoute, RouteSchedule, Hotel, RoomCategory, HolidayPackage, PromoCode
Weak Entities	6	ScheduleClass, ScheduleDeparture, PackageDeparture, RoomAvailability, Passenger, PackageTraveller
Feature Entities	4	Insurance, CancellationRefund, Review, DemandTag
Superclass	1	Booking
Subclasses	3	TransportBooking, HotelBooking, PackageBooking
Binary Relationships	14	operates, has_schedule, has_class, lists, has_rooms, tracked_on, offers, has_departure, applies, writes, covered_by, cancelled_as, reviewed_by, has_passenger
Ternary Relationships	4	generates, books_transport, books_hotel, books_package
Aggregation boxes	1	Booking cluster
Generalizations	1	Booking → 3 subclasses